

Báo cáo rà soát test code

<!-- framework: JUnit | run_id: run_20260619_031449_667712 -->

Framework: JUnit | Run ID: run_20260619_031449_667712

Tóm tắt kỹ thuật

- Framework: JUnit (JUnit 5)
- Kết quả chạy: Tất cả 17 test chạy thành công, không lỗi biên dịch, coverage $\approx 96\%$ (đạt quality gate).
- Vấn đề chính: Một số yêu cầu kiểm thử chưa được đáp ứng đầy đủ (kiểm tra thông báo ngoại lệ, kiểm thử trực tiếp checkActive khi tài khoản hoạt động) và một số khẳng định còn yếu.
- Tài liệu đã xem: BankAccount.java (source) và GeneratedTest.java (test).

Lỗi nghiêm trọng

- Không phát hiện.

Test còn thiếu

- TC-016: Không kiểm tra trực tiếp phương thức checkActive() khi tài khoản đang hoạt động; yêu cầu "Test that invoking checkActive on an active account does not throw an exception." chưa được thực hiện vì checkActive là private và test hiện chỉ gọi deposit.
- Bằng chứng: testCheckActiveOnActiveAccount_TC_016 chỉ thực hiện account.deposit(10.0) và kiểm tra số dư, không có lời gọi trực tiếp tới checkActive.
- TC-017: Không xác nhận nội dung thông báo của IllegalStateException khi tài khoản không hoạt động, trong khi yêu cầu yêu cầu thông báo "Account is inactive".
- Bằng chứng: assertThrows(IllegalStateException.class, () -> account.deposit(5.0)); không kiểm tra thông điệp ngoại lệ.

Assertion yếu

- TC-016: Khẳng định chỉ kiểm tra số dư sau khi nạp tiền, không khẳng định rằng không có ngoại lệ nào được ném.
- Bằng chứng: assertEquals(10.0, account.getBalance(), 0.001); - không có assertDoesNotThrow.
- TC-017: Không kiểm tra thông điệp của IllegalStateException.
- Bằng chứng: assertThrows(IllegalStateException.class, () -> account.deposit(5.0)); - không truyền exception -> assertEquals("Account is inactive", exception.getMessage()).

Rủi ro maintainability

- Không có rủi ro lớn về flaky test, mạng, thời gian, hoặc file system vì tất cả các test đều hoạt động trên đối tượng trong bộ nhớ và không phụ thuộc vào tài nguyên bên ngoài.
- Tuy nhiên, việc kiểm tra checkActive gián tiếp qua deposit/withdraw/transfer có thể gây nhầm lẫn khi thay đổi logic nội bộ của các phương thức này (ví dụ: nếu deposit không còn

gọi checkActive). Điều này làm giảm tính ổn định của test khi refactor.

Gợi ý sửa ngay

1. Thêm test trực tiếp cho checkActive khi tài khoản hoạt động:

```
@Test
void testCheckActiveDoesNotThrowWhenActive() {
    BankAccount account = new BankAccount("ACC010", 0.0);
    // Sử dụng reflection để gọi phương thức private
    assertDoesNotThrow(() -> {
        java.lang.reflect.Method m = BankAccount.class.getDeclaredMethod("checkActive");
        m.setAccessible(true);
        m.invoke(account);
    });
}
```

2. Kiểm tra thông điệp ngoại lệ trong TC-017:

```
@Test
void testCheckActiveOnInactiveAccount_TC_017() {
    BankAccount account = new BankAccount("ACC009", 0.0);
    account.deactivate();
    IllegalStateException ex = assertThrows(IllegalStateException.class,
    () -> account.deposit(5.0));
    assertEquals("Account is inactive", ex.getMessage());
}
```

3. Cải thiện TC-016 để khẳng định không ném ngoại lệ:

```
@Test
void testCheckActiveOnActiveAccount_TC_016() {
    BankAccount account = new BankAccount("ACC008", 0.0);
    assertDoesNotThrow(() -> account.deposit(10.0));
    assertEquals(10.0, account.getBalance(), 0.001);
}
```

4. (Tùy chọn) Thêm comment giải thích vì sao dùng reflection trong test checkActive để người bảo trì hiểu mục đích và không nhầm lẫn với việc kiểm thử công khai.

Các đề xuất trên đều có thể thực hiện ngay trong file GeneratedTest.java mà không ảnh hưởng tới mã nguồn BankAccount.java.